

4) Utilizzando NumPy e OpenCV, implementare in Python la funzione *esercizio(img)* che riceve un'immagine grayscale *img* (con un byte per pixel) e deve eseguire le seguenti operazioni:

- 1) Calcolare, per ogni pixel dell'immagine *img*, la media dei pixel in un intorno 7x7. Impostare a zero 3 pixel lungo tutti i bordi nell'immagine risultato.
- 2) Calcolare l'immagine *Diff* in cui ciascun pixel è il valore assoluto della differenza fra il pixel corrispondente nell'immagine originale e il valore determinato al passo precedente.
- 3) Eseguire l'operazione di contrast stretching su *Diff*.
- 4) Binarizzare *Diff* utilizzando come soglia globale il valore 128.
- 5) Etichettare le componenti connesse del risultato ottenuto al passo precedente; eliminare quindi le componenti connesse con area inferiore a 15 pixel.
- 6) Restituire come output un'immagine grayscale in cui i pixel appartenenti alle componenti connesse rimanenti dopo il passo precedente hanno valore 255, mentre tutti gli altri 0.

<pre>import numpy as np import cv2 as cv def esercizio(img): avg_img = cv.boxFilter(img, -1, (7, 7)) avg_img[:3], avg_img[:, :3], avg_img[-3:], avg_img[:, -3:] = 0, 0, 0, 0 Diff = np.abs(img - avg_img) v_min, v_max = Diff.min(), Diff.max() if (v_max > v_min): Diff = (255 * (Diff.astype(float)-v_min)/(v_max-v_min)).astype(np.uint8) _, bin_img = cv.threshold(Diff, 128, 255, cv.THRESH_BINARY) n, cc, stats, _ = cv.connectedComponentsWithStats(bin_img) toRemove = np.argwhere(stats[:, cv.CC_STAT_AREA] < 15) return np.where(np.isin(cc, toRemove), 0, bin_img).astype(np.uint8)</pre>	
	<i>Promemoria: alcune funzioni che potrebbero essere utili</i> cv.threshold(src, thresh, maxval, type) -> retval, dst cv.boxFilter(src, ddepth, ksize) -> dst connectedComponentsWithStats(image) -> retval, labels, stats, centroids <i>Promemoria: alcune costanti che potrebbero essere utili</i> cv.THRESH_BINARY cv.CC_STAT_AREA